

PRACTICAL WEEK-11

1 . Develop a multithreaded counter application using POSIX threads (`pthread_create`, `pthread_join`). Demonstrate a race condition by allowing multiple threads to update a shared variable concurrently.

Aim : To create multiple threads using POSIX threads and demonstrate a race condition when multiple threads concurrently modify a shared counter.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

void *increment_counter(void *arg)
{
    int thread_id = *(int *)arg;

    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    printf("Thread %d completed.\n", thread_id);

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int thread_id[NUM_THREADS];

    printf("Multithreaded Counter Application\n");
    printf("-----\n");

    printf("Initial counter: %d\n", counter);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        thread_id[i] = i + 1;

        if (pthread_create(&threads[i],
                           NULL,
                           increment_counter,
                           &thread_id[i]) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }
}
```

GNU nano 7.2

```
int thread_id[NUM_THREADS];

printf("Multithreaded Counter Application\n");
printf("-----\n");

printf("Initial counter: %d\n", counter);

for (int i = 0; i < NUM_THREADS; i++)
{
    thread_id[i] = i + 1;

    if (pthread_create(&threads[i],
                      NULL,
                      increment_counter,
                      &thread_id[i]) != 0)
    {
        perror("pthread_create");
        return 1;
    }
}

for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_join(threads[i], NULL) != 0)
    {
        perror("pthread_join");
        return 1;
    }
}

printf("\nExpected counter: %d\n",
      NUM_THREADS * ITERATIONS);

printf("Actual counter:   %d\n", counter);

if (counter != NUM_THREADS * ITERATIONS)
{
    printf("Race condition detected!\n");
}
else
{
    printf("Counter reached expected value.\n");
}

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

Output :

```
root@keerthana: ~/OSSP
root@keerthana:~/OSSP# cd ~/OSSP
root@keerthana:~/OSSP# mkdir -p ~/OSSP
root@keerthana:~/OSSP# nano thread_counter.c
root@keerthana:~/OSSP# gcc -Wall -Wextra -pthread thread_counter.c -o thread_counter
root@keerthana:~/OSSP# ./thread_counter
Multithreaded Counter Application
-----
Initial counter: 0
Thread 1 completed.
Thread 2 completed.
Thread 3 completed.
Thread 4 completed.

Expected counter: 400000
Actual counter: 343309
Race condition detected!
root@keerthana:~/OSSP# ./thread_counter
Multithreaded Counter Application
-----
Initial counter: 0
Thread 1 completed.
Thread 2 completed.
Thread 3 completed.
Thread 4 completed.

Expected counter: 400000
Actual counter: 263397
Race condition detected!
root@keerthana:~/OSSP# ./thread_counter
Multithreaded Counter Application
-----
Initial counter: 0
Thread 2 completed.
Thread 1 completed.
Thread 3 completed.
Thread 4 completed.

Expected counter: 400000
Actual counter: 229476
Race condition detected!
root@keerthana:~/OSSP# ./thread_counter
Multithreaded Counter Application
-----
Initial counter: 0
Thread 2 completed.
Thread 1 completed.
Thread 4 completed.
Thread 3 completed.

Expected counter: 400000
Actual counter: 179300
Race condition detected!
root@keerthana:~/OSSP# ./thread_counter
Multithreaded Counter Application
-----
Initial counter: 0
Thread 1 completed.
Thread 4 completed.
Thread 2 completed.
Thread 3 completed.

Expected counter: 400000
Actual counter: 214006
Race condition detected!
root@keerthana:~/OSSP# ps -elf | grep thread_counter
```

2 . Modify the previous multithreaded counter program using mutex locks (`pthread_mutex_lock()` and `pthread_mutex_unlock()`) and compare the output with the race-condition version. Analyze the impact of synchronization.

Aim : To eliminate the race condition in a multithreaded counter application using a POSIX mutex and compare synchronized execution with unsynchronized execution.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;
pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    int thread_id = *(int *)arg;
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    printf("Thread %d completed.\n", thread_id);
    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int thread_id[NUM_THREADS];

    printf("Multithreaded Counter with Mutex\n");
    printf("-----\n");

    if (pthread_mutex_init(&lock, NULL) != 0)
    {
        perror("Mutex initialization failed");
        return 1;
    }

    printf("Initial counter: %d\n", counter);
    for (int i = 0; i < NUM_THREADS; i++)
    {
        ^G Help          ^O Write Out      ^W Where Is      ^F
        ^X Exit          ^R Read File      ^_ Replace       ^L
```

GNU nano 7.2

```
printf("Initial counter: %d\n", counter);

for (int i = 0; i < NUM_THREADS; i++)
{
    thread_id[i] = i + 1;

    if (pthread_create(&threads[i],
                      NULL,
                      increment_counter,
                      &thread_id[i]) != 0)
    {
        perror("pthread_create");
        pthread_mutex_destroy(&lock);
        return 1;
    }
}

for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_join(threads[i], NULL) != 0)
    {
        perror("pthread_join");
        pthread_mutex_destroy(&lock);
        return 1;
    }
}

printf("\nExpected counter: %d\n",
      NUM_THREADS * ITERATIONS);

printf("Actual counter:   %d\n", counter);

if (counter == NUM_THREADS * ITERATIONS)
{
    printf("Synchronization successful.\n");
    printf("Race condition prevented.\n");
}
else
{
    printf("Unexpected result.\n");
}

pthread_mutex_destroy(&lock);

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^\ **Replace**

^K Cu
^U Pa

Output :

```
root@keerthana: ~/OSSP  ×  +  ∨

root@keerthana:~/OSSP# nano thread_counter_mutex.c
root@keerthana:~/OSSP# gcc -Wall -Wextra -pthread thread_counter_mutex.c -o thread_counter_mutex
root@keerthana:~/OSSP# ./thread_counter_mutex
Multithreaded Counter with Mutex
-----
Initial counter: 0
Thread 4 completed.
Thread 3 completed.
Thread 1 completed.
Thread 2 completed.

Expected counter: 400000
Actual counter: 400000
Synchronization successful.
Race condition prevented.
root@keerthana:~/OSSP# ./thread_counter_mutex
Multithreaded Counter with Mutex
-----
Initial counter: 0
Thread 2 completed.
Thread 4 completed.
Thread 1 completed.
Thread 3 completed.

Expected counter: 400000
Actual counter: 400000
Synchronization successful.
Race condition prevented.
root@keerthana:~/OSSP# ./thread_counter_mutex
Multithreaded Counter with Mutex
-----
Initial counter: 0
Thread 1 completed.
Thread 3 completed.
Thread 2 completed.
Thread 4 completed.

Expected counter: 400000
Actual counter: 400000
Synchronization successful.
Race condition prevented.
root@keerthana:~/OSSP# ./thread_counter_mutex
Multithreaded Counter with Mutex
-----
Initial counter: 0
Thread 2 completed.
Thread 1 completed.
Thread 3 completed.
Thread 4 completed.

Expected counter: 400000
Actual counter: 400000
Synchronization successful.
Race condition prevented.
root@keerthana:~/OSSP# |
```

